

LinkUp

Realtime Architecture — How Chat, Notifications, and Audio/Video Calls Work Internally

A step-by-step technical guide with sample code

Stack: .NET 10 Web API · SignalR · WebRTC · Angular 22

Modular monolith · JWT-authenticated hubs

Table of Contents

1. Architecture Overview
2. SignalR Fundamentals (connection, groups, auth)
3. Realtime Chat (presence, messages, typing, receipts)
4. Realtime Notifications (cross-module, settings-aware)
5. Audio & Video Calls (WebRTC + SignalR signaling)
6. Ringtone, Call-End Sync & Noise Cancellation
7. Client Wiring Summary

1. Architecture Overview

LinkUp is a modular monolith. Request/response data flows through `Controller → Manager → DbContext → PostgreSQL`. Anything that must reach a user *without them asking* — a new message, a like, an incoming call — is pushed over **SignalR** (a persistent WebSocket connection). Peer-to-peer audio/video media itself never touches the server; only its **signaling** does.

Concern	Transport	Hub / Mechanism
Chat messages, presence, typing, receipts	SignalR (server relay)	<code>ChatHub</code>
Notifications (likes, comments, friend requests)	SignalR (server push)	<code>NotificationHub</code>
Call ringing + WebRTC signaling (SDP/ICE)	SignalR (server relay)	<code>VideoCallHub</code>
Actual audio/video media	WebRTC (peer-to-peer)	Browser <code>RTCPeerConnection</code>

Key idea: SignalR is the "nervous system." For chat and notifications it *carries the payload*. For calls it only *coordinates the handshake* — once two browsers have exchanged connection details, the audio and video stream directly between them.

2. SignalR Fundamentals

2.1 Connecting with a JWT

Every hub is `[Authorize]`. The Angular client passes the JWT via `accessTokenFactory`. SignalR upgrades to WebSockets and authenticates the connection from that token.

```
// video-call-hub.service.ts - client connect (idempotent)
this.hub = new HubConnectionBuilder()
  .withUrl(`${environment.hubUrl}/videocall`, { accessTokenFactory: () => token ?? '' })
  .withAutomaticReconnect()
  .build();
await this.hub.start();
```

2.2 Groups — how the server targets the right people

SignalR **groups** are the routing primitive. LinkUp uses two conventions:

- **Personal group** — every connection joins a group named by the user's own id. This lets the server push to "that user, wherever they are connected" (e.g. ring a call, deliver a notification).
- **Room group** — a connection joins a group named by a `chatId` or `callId` to receive everything happening in that conversation/call.

```
// On connect: join a personal group so we can be reached directly
await Groups.AddToGroupAsync(Context.ConnectionId, userId.ToString());

// Server later pushes straight to that user on any device:
await hubContext.Clients.Group(userId.ToString())
  .SendAsync("ReceiveNotification", dto);
```

Gotcha (fixed in code): SignalR sends **positional arguments**, not a single object. A server call `SendAsync("UserTyping", chatId, userId, isTyping)` must be received as `.on('UserTyping', (chatId, userId, isTyping) => ...)` — the argument order must match exactly.

3. Realtime Chat ChatHub

3.1 Presence — online / offline / last-seen

When a socket connects, the hub flips the user online, joins their personal group and **all their active chat groups**, and broadcasts presence to everyone else. On disconnect it records `LastSeen` and broadcasts offline.

```
// ChatHub.cs
public override async Task OnConnectedAsync()
{
    var user = await userManager.FindByIdAsync(CurrentUserId.ToString());
    user.IsOnline = true; user.LastSeen = DateTime.UtcNow;
    await userManager.UpdateAsync(user);
    await Clients.Others.SendAsync("UserOnline", CurrentUserId);

    await Groups.AddToGroupAsync(Context.ConnectionId, CurrentUserId.ToString());
    var chatIds = await db.ChatParticipants
        .Where(cp => cp.UserId == CurrentUserId && cp.IsActive)
        .Select(cp => cp.ChatId).ToListAsync();
    foreach (var chatId in chatIds)
        await Groups.AddToGroupAsync(Context.ConnectionId, chatId.ToString());
    await base.OnConnectedAsync();
}
```

3.2 Sending a message — step by step

1. Client POSTs the message to the REST API (`POST /chats/messages`).
2. `ChatManager` persists it; marks it **Delivered** if any recipient is online.
3. Manager broadcasts `ReceiveMessage` to the whole chat group (via `IHubContext`).
4. Every connected member — *including the sender* — receives it in real time.
5. The client de-duplicates by message id (the sender already appended it from the REST reply).

```
// ChatManager.SendMessageAsync – persist then push over SignalR
await db.SaveChangesAsync(ct);
var messageDto = await BuildMessageDtoAsync(message, senderId, ct);
await hubContext.Clients.Group(dto.ChatId.ToString())
    .SendAsync("ReceiveMessage", messageDto, ct);
```

```
// chat.component.ts – client side: dedup so a message never shows twice
this.chatHub.message$.subscribe(msg => {
    if (msg.chatId === this.activeChat()?.id) this.appendMessage(msg); // guarded by id
});

private appendMessage(msg: MessageDto): void {
    this.messages.update(m => m.some(x => x.id === msg.id) ? m : [...m, msg]);
}
```

Why the dedup matters: the server broadcasts to the entire group *including the sender*. Without de-duplication by id, the sender's own message would appear twice — very visible in group chats.

3.3 Typing indicator & delivery/read receipts

These are lightweight hub methods that relay only to `OthersInGroup` (never echo to the actor):

```
// ChatHub.cs
public async Task SendTypingIndicator(string chatId, bool isTyping) =>
    await Clients.OthersInGroup(chatId).SendAsync("UserTyping", Guid.Parse(chatId), CurrentUserId, isTyping);

public async Task MarkAsRead(string messageId, string chatId) {
    await chatManager.MarkReadAsync(Guid.Parse(messageId), CurrentUserId); // persist
    await Clients.OthersInGroup(chatId).SendAsync("MessageRead", Guid.Parse(messageId), CurrentUserId);
}
```

4. Realtime Notifications NotificationHub

Notifications are generated **cross-module**: the Friend, Reaction, Comment and Chat managers inject `INotificationManager` and call one method. It persists the notification, **honors the recipient's per-category settings**, and pushes it to the recipient's personal group along with an updated unread count.

1. A user likes a post → `ReactionManager` calls `CreateNotificationAsync(...)`.
2. Manager checks the recipient's `NotificationSettings` — disabled categories are silently dropped.
3. It saves the row, enriches it with sender name/avatar, then pushes.
4. Recipient (on any device) instantly receives `ReceiveNotification` + `NotificationCountUpdated`.

```
// NotificationManager.CreateNotificationAsync (abridged)
if (!await IsTypeEnabledAsync(dto.RecipientId, dto.Type, ct))
    return ...; // setting disabled → suppress

db.Notifications.Add(notification);
await db.SaveChangesAsync(ct);
var notificationDto = mapper.Map<NotificationDto>(notification); // + sender info

var group = dto.RecipientId.ToString();
await hubContext.Clients.Group(group).SendAsync("ReceiveNotification", notificationDto, ct);
await hubContext.Clients.Group(group).SendAsync("NotificationCountUpdated", await GetUnreadCountAsync(
```

```
// NotificationHub.cs — just parks each connection in the user's personal group
public override async Task OnConnectedAsync() {
    var userId = Context.User!.FindFirstValue(ClaimTypes.NameIdentifier!);
    await Groups.AddToGroupAsync(Context.ConnectionId, userId);
    await base.OnConnectedAsync();
}
```

Design note: "New message" and "video call" are intentionally *not* notification-feed entries — chat has its own unread badge and calls use the ringing hub, so they aren't duplicated in the feed.

5. Audio & Video Calls VideoCallHub + WebRTC

A call has two layers: **signaling** (over SignalR) to set up the connection, and the **media** (over WebRTC) that flows peer-to-peer. The server only relays signaling; it never sees audio or video.

5.1 The WebRTC handshake in plain terms

Term	What it is
SDP offer / answer	A text description of "here are the codecs and media I can do." Caller offers, callee answers.
ICE candidate	A possible network path (IP/port) to reach a peer. Both sides trickle these to each other.
STUN server	Helps a browser discover its public address behind NAT (<code>stun:stun.l.google.com:19302</code>).
RTCPeerConnection	The browser object that manages the P2P media connection.

5.2 End-to-end call flow (1:1)

- 1. Ring** — caller invokes `StartCall(callId, [calleeId], "video", false)`. The hub sends `CallRinging` to the callee's personal group. The callee's device shows a banner and plays a ringtone.
- 2. Answer** — callee accepts and navigates to the call screen; both call `JoinCall(callId)`.
- 3. Discover peers** — the hub replies to the newcomer with `ExistingParticipants` and tells the others `ParticipantJoined`. The newcomer creates an **offer** to each existing peer.
- 4. Exchange SDP** — offer → `SendSdpOffer` relayed to the target; the target sets it, creates an **answer**, relays it back via `SendSdpAnswer`.
- 5. Exchange ICE** — as candidates are gathered they are relayed with `SendIceCandidate`. Candidates arriving before the remote description is set are **buffered**, then flushed.
- 6. Connected** — once a peer connection reports `connected`, audio/video flow directly, P2P.

5.3 Server: pure relay, tagged with the sender

```
// VideoCallHub.cs – ring, then relay SDP/ICE to a specific user
public async Task StartCall(string callId, string[] inviteeIds, string mediaType, bool isGroup) {
    foreach (var id in inviteeIds)
        await Clients.Group(id).SendAsync("CallRinging", callId, Uid, mediaType, isGroup);
    // ... best-effort call-history persistence ...
}

public async Task JoinCall(string callId) {
    var members = _rooms.GetOrAdd(callId, _ => new());
    var existing = members.Values.Where(u => u != Uid).Distinct().ToArray();
    members[Context.ConnectionId] = Uid;
    await Groups.AddToGroupAsync(Context.ConnectionId, callId);
    await Clients.Caller.SendAsync("ExistingParticipants", existing); // newcomer offers to each
    await Clients.OthersInGroup(callId).SendAsync("ParticipantJoined", Uid);
}

// SDP/ICE are relayed to one user and tagged with the sender (Uid) + callId
public Task SendSdpOffer(string callId, string targetUserId, string sdp) =>
    Clients.Group(targetUserId).SendAsync("SdpOfferReceived", callId, sdp, Uid);
public Task SendIceCandidate(string callId, string targetUserId, string candidate) =>
    Clients.Group(targetUserId).SendAsync("IceCandidateReceived", callId, candidate, Uid);
```


5.4 Client: building the peer connection (mesh)

```
// video-call.component.ts — one RTCPeerConnection per remote peer
private ensurePeer(userId: string): PeerConn {
  const pc = new RTCPeerConnection({ iceServers: [{ urls: 'stun:stun.l.google.com:19302' }] });
  this.localStream?.getTracks().forEach(track => pc.addTrack(track, this.localStream!));
  pc.ontrack = e => this.showRemoteStream(userId, e.streams[0]);
  pc.onicecandidate = e => { if (e.candidate) this.hub.sendIceCandidate(this.callId, userId, e.candidate); };
  return { pc, pendingCandidates: [], remoteSet: false };
}

private async makeOffer(userId: string) {
  const { pc } = this.ensurePeer(userId);
  const offer = await pc.createOffer();
  await pc.setLocalDescription(offer);
  this.hub.sendSdpOffer(this.callId, userId, offer);
}

private async handleOffer(userId: string, sdp: RTCSessionDescriptionInit) {
  const entry = this.ensurePeer(userId);
  await entry.pc.setRemoteDescription(new RTCSessionDescription(sdp));
  entry.remoteSet = true; await this.flush(userId); // apply buffered ICE
  const answer = await entry.pc.createAnswer();
  await entry.pc.setLocalDescription(answer);
  this.hub.sendSdpAnswer(this.callId, userId, answer);
}

private async addIce(userId: string, candidate: RTCIceCandidateInit) {
  const entry = this.ensurePeer(userId);
  if (entry.remoteSet) await entry.pc.addIceCandidate(new RTCIceCandidate(candidate));
  else entry.pendingCandidates.push(candidate); // buffer until remote description is set
}
```

Why buffer ICE? ICE candidates can arrive before the SDP answer/offer is applied. Adding a candidate before `setRemoteDescription` throws, so early candidates are queued and flushed once the remote description is set — this is the single most common source of "call connects but no media" bugs.

6. Ringtone, Call-End Sync & Noise Cancellation

6.1 Ringtone on the callee's device

The callee's `CallService` starts a looping ringtone the moment `CallRinging` arrives. It is generated with the **Web Audio API** (no audio file needed, so no asset/CSP dependency) and stops on accept, decline, or timeout.

```
// call.service.ts - two-tone ring generated with oscillators, looped every 3s
private startRingtone(): void {
  this.audioCtx = this.audioCtx ?? new AudioContext();
  const burst = () => { tone(480, 0); tone(440, 0.5); }; // classic ring cadence
  burst();
  this.ringInterval = setInterval(burst, 3000);
  this.ringTimeout = setTimeout(() => { // missed call after 35s
    const call = this.incomingCall();
    if (call) this.hub.declineCall(call.callId, call.callerId); // tell the caller too
    this.stopRingtone(); this.incomingCall.set(null);
  }, 35000);
}
```

6.2 Ending the call on BOTH sides

Decline — callee taps decline → `DeclineCall` → `CallDeclined` to the caller → caller's screen ends.

No answer — the ring times out (35s) and sends the same `DeclineCall`, so the caller ends too.

Safety net — the caller's call screen also self-ends after 45s if it never connects (callee offline).

```
// video-call.component.ts
this.hub.callDeclined$.subscribe(({ callId }) => {
  if (callId === this.callId && !this.isGroup() && this.conns.size === 0) this.endLocally();
});
// caller-side no-answer fallback
if (!this.isGroup())
  this.noAnswerTimer = setTimeout(() => { if (this.callStatus() === 'connecting') this.endLocally(); }
```

6.3 Noise cancellation

Audio is captured with the browser's built-in DSP enabled — echo cancellation, noise suppression and automatic gain control — instead of a bare `audio: true`.

```
// video-call.component.ts - initLocalMedia()
const constraints = {
  video: this.isVideoCall(),
  audio: { echoCancellation: true, noiseSuppression: true, autoGainControl: true }
};
this.localStream = await navigator.mediaDevices.getUserMedia(constraints);
```

7. Client Wiring Summary

Server event (SignalR)	Client handler	Effect
ReceiveMessage	chatHub.message\$	Append to open chat (dedup by id), bump unread badge
UserOnline/UserOffline	chatHub.userOnline\$	Update presence dots + last-seen
UserTyping	chatHub.userTyping\$	Show "typing..." indicator
MessageDelivered/Read	chatHub.message*\$	Update ✓ / ✓✓ ticks
ReceiveNotification	notification service	Prepend to feed, increment count
CallRinging	callSvc.incomingCall	Show banner + play ringtone
SdpOffer/Answer/Ice... Received	video-call component	Drive the WebRTC handshake
ParticipantJoined/Left	video-call component	Add/remove a remote video tile

In one sentence: SignalR keeps a JWT-authenticated socket open per user; the server pushes chat and notifications straight into per-user/per-room groups, and for calls it only brokers the WebRTC handshake so audio and video can flow directly between the two browsers.